

Health Buddy — 产品全框架

无穿戴健康监测桌宠 · 全栈 TypeScript · Monorepo · 六层架构
来源: health-buddy-framework.canvas.tsx 内容导出

一、用户体验流

用户从首次打开网页，到获得完整健康监测体验的完整旅程。

#	步骤	用户操作 / 感受	核心技术	阶段
1	打开网页	看到坡屋顶小树屋动画 像进入一个温馨的小世界	React Router → TreeHouseEntry 页	Phase 1
2	点击门	门打开，进入选宠物界面 惊喜感，有仪式感	Framer Motion rotateY 门动画	Phase 1
3	选择小鸟	种类 / 毛色 / 性别 / 名字 / 性格 强烈的角色代入感	PetSelection 页 → Zustand + localStorage	Phase 1
4	进入树屋	小鸟出现并说第一句关怀话 暖心，想继续养	Dashboard + mock 健康数据 + 台词库	Phase 1
5	绑定手机	扫码 / 同账号，授权步数和运动状态 感觉小鸟更懂自己了	React Native + expo-sensors + WebSocket	Phase 2
6	安装桌面客户端	Electron 悬浮窗，键鼠信号实时采集 小鸟常驻桌面，润物细无声	Electron + uiohook-napi + powerMonitor	Phase 2
7	公司电脑场景	打开网页心跳会话，手机作锚点 不用装软件，体验连续	apps/server workSessions Hono API	Phase 2
8	小鸟更智能	LLM 生成个性化台词，记住你的习惯 像真正的私人健康伙伴	packages/llm-client → DeepSeek API	Phase 3

二、系统架构（六层）

各层职责分离，通过 packages/shared-types 统一类型契约跨端通信。

L1 数据采集层

信号 / 功能	技术实现	备注
桌面键鼠强度	uihook-napi (Electron)	每分钟聚合强度，不记录内容
锁屏 / 空闲	Electron powerMonitor	observed_zero vs missing 区分
手机步数 / 运动	expo-sensors + HealthKit / Google Fit	步数增量，运动状态
手机亮屏事件	React Native AppState	unlock / lock / interactive
近距离信号	同 WiFi / 蓝牙 RSSI	判断用户是否在自己电脑旁
Web 心跳会话	Browser tab + fetch / WebSocket	公共电脑轻量接入
摄像头心率 rPPG	Python 子进程 + IPC	可选，用户主动开启

L2 决策仲裁层

信号 / 功能	技术实现	备注
多状态打分器	packages/decision-engine	候选状态打分，不输出单一二值
观测覆盖率	ObservationCoverage	coverageScore 低时输出 unknown
置信度门控	ConfidenceLevel: high/medium/low	驱动宠物提醒等级
信号区分	SignalQuality 枚举	observed_zero / missing / stale / disabled

L3 健康分析层

信号 / 功能	技术实现	备注
久坐检测	packages/health-engine	连续工作时长统计
疲劳指数	规则引擎 (Phase 3 → LLM)	0-100 打分
睡眠推断	无活动 + 时段推断	依赖夜间手机 / 桌面信号
心率估算	rPPG Python 子进程 → IPC	可选，不影响主流程

L4 行为映射层

信号 / 功能	技术实现	备注
行为状态机	packages/pet-engine	idle / yawn / stretch / worried / happy
触发器匹配	优先级队列 + 置信度门控	high→直接提醒，low→安静陪伴

信号 / 功能	技术实现	备注
性格过滤器	personality → 台词库	温柔 / 傲娇 / 开朗 / 沉稳 / 调皮
LLM 文案生成	packages/llm-client (Phase 3)	DeepSeek API 封装

L5 宠物 UI 层 & L6 树屋仪表盘

信号 / 功能	技术实现	备注
树屋网页	React 18 + Vite + Tailwind CSS	Phase 1 首发, web-first
帧动画播放器	CSS 帧动画 + Framer Motion	SVG 占位 → Phase 2 PNG 序列帧
说话气泡	Framer Motion + 打字机效果	SpeechBubble 组件
Electron 悬浮窗	Electron BrowserWindow	透明窗口悬浮桌面
历史健康图表	React + SVG 折线图	7天 / 30天趋势
宠物配置中心	LocalStorage / Server DB	种类 / 毛色 / 性格 / 名字

跨层数据类型契约

数据流	类型	文件
L1 → L2	KeyMouseSnapshot, PhoneMotionEvent, ProximitySignal, WebWorkSession	shared-types/index.ts
L2 → L3	ActivityDecision (state + confidence + observationCoverage)	shared-types/index.ts
L3 → L4	HealthSnapshot (fatigueScore, sedentaryMinutes, hydrationReminderDue)	shared-types/index.ts
L4 → L5	PetTrigger (kind + priority + payload)	shared-types/index.ts
L5 ↔ L6	DailyHealthReport + PetConfig + UserPreferences	shared-types/index.ts
置信度门控	high→直接提醒 · medium→询问式 · low→安静陪伴 · unknown→不计久坐	pet-engine/index.ts

三、技术栈

全栈 TypeScript, Yarn Workspaces Monorepo。6 层系统 · 4 应用端 · 5 共享 packages · 100% TypeScript。

应用 / 包	框架	工具 1	工具 2	备注
网页前端	React 18 + TypeScript	Vite	Tailwind + Framer Motion	Zustand
桌面端	Electron + TypeScript	uiohook-napi	powerMonitor	复用 Web 组件
手机端	React Native + Expo	expo-sensors	HealthKit / Google Fit	AppState
后端服务	Hono (TypeScript)	better-sqlite3	Zod 数据校验	WebSocket
决策引擎	packages/decision-engine	多状态打分器	ObservationCoverage	SignalQuality
健康分析	packages/health-engine	规则引擎	Phase 3 → LLM	HealthSnapshot
宠物引擎	packages/pet-engine	行为状态机	性格过滤器	PetTrigger 队列
LLM 客户端	packages/llm-client (P3)	DeepSeek API	SystemPrompt	对话上下文管理

后端 API 端点 (Hono TypeScript)

端点	方法	功能	阶段
/api/mobile/signals	POST	接收手机端信号批次	Phase 2
/api/work-sessions	POST	公共电脑开启 Web 工作会话	Phase 2
/api/work-sessions/heartbeat	POST	心跳保活 (30 秒一次)	Phase 2
/api/work-sessions/:uid/active	GET	查询当前活跃会话	Phase 2
/api/health/snapshot	GET	获取最新健康快照	Phase 2
/api/health/daily/:date	GET	获取每日健康报告	Phase 2
/api/pet/config	GET / PUT	读取 / 保存宠物配置	Phase 1
/api/user/preferences	POST	更新用户偏好	Phase 3

四、实现路线图

Phase 1 — Web MVP (第 1–6 周) 目标：完整体验闭环，数据用 mock，让用户先养起来。	Phase 2 — 多端数据接入 (第 7–16 周) 目标：真实信号替换 mock，手机和电脑联动。
---	---

- TreeHouseEntry 树屋开场动画
- PetSelection 选宠物四步向导
- Dashboard + 健康卡片 + SpeechBubble
- mock/healthData.ts + petBehaviorMapping.ts

交付: 打开网页 → 选好小鸟 → 小鸟说一句符合性格的关怀话

- Electron 悬浮窗 + keyMouse + powerMonitor
- React Native 步数 + 运动状态上报
- Python → TypeScript Hono 迁移
- decision-engine 接入真实信号

交付: 小鸟感知真实久坐 / 步数 / 疲劳, 提醒有根据

Phase 3 — AI 深化 (第 11–16 周, 与 P2 并行)

目标: 小鸟更聪明, 对话有个性, 提醒有温度。

- packages/llm-client DeepSeek API
- health-engine LLM 生成 aiSummary
- rPPG Python 子进程 → IPC
- 历史图表 + 用户偏好 + 宠物调教

交付: 高置信度直接提醒, 低置信度安静陪伴, unknown 不计久坐

团队分工建议

角色	主要职责	核心产出
前端 (1–2 人)	apps/web/ Phase 1 全部	TreeHouseEntry / PetSelection / Dashboard / PetSprite
移动端 (1 人)	apps/mobile/ 手机采集、绑定、上报	lowFrictionSignals.ts / signalUpload.ts
后端 (1人)	apps/server/ Hono + SQLite + WebSocket	routes/*.ts / db/sqlite.ts
算法 (1–2 人)	decision / health / pet-engine	arbitrate() / HealthSnapshot / PetTrigger

五、隐私与决策

核心原则: 不是数据越多越好, 而是「正确时刻的轻柔干预」。用户不应感到被监视。

数据采集权限分层

数据类型	采集策略	技术说明
桌面活跃 / 空闲 / 锁屏	默认开启	Electron powerMonitor
每分钟键鼠活动强度	默认开启	uiohook-napi 聚合后丢弃原始值
手机亮屏 / 锁屏事件	默认开启	AppState, 不读通知内容
步数增量 / 运动状态	默认开启	读增量, 不读位置
同 WiFi / 蓝牙近距离	默认开启	仅判断是否在自己电脑旁
应用使用统计 / rPPG	用户主动开启	UsageStats / Screen Time / 本地推断
按键内容 / 截图 / 位置 / 消息	永不采集	—

宠物提醒置信度降级策略

置信度	行为	说明
高置信度	直接提醒	明确观测到久坐 / 疲劳 / 缺水, 直接说话
中置信度	询问式提醒	「你是不是累了？」不强推, 留余地
低置信度	安静陪伴	不打扰, 只展示 idle 动作
unknown	不计入统计	信号不足时不累积久坐时长, 避免误判

决策层 SignalQuality 区分

类型	含义
observed_positive	明确观测到活动: 键鼠输入 / 手机亮屏 / 步数增加
observed_zero	采集器在线但观测到 0: 桌面解锁但本分钟无键鼠活动
missing	信号缺失: 未装客户端 / 未授权 / 网络断开